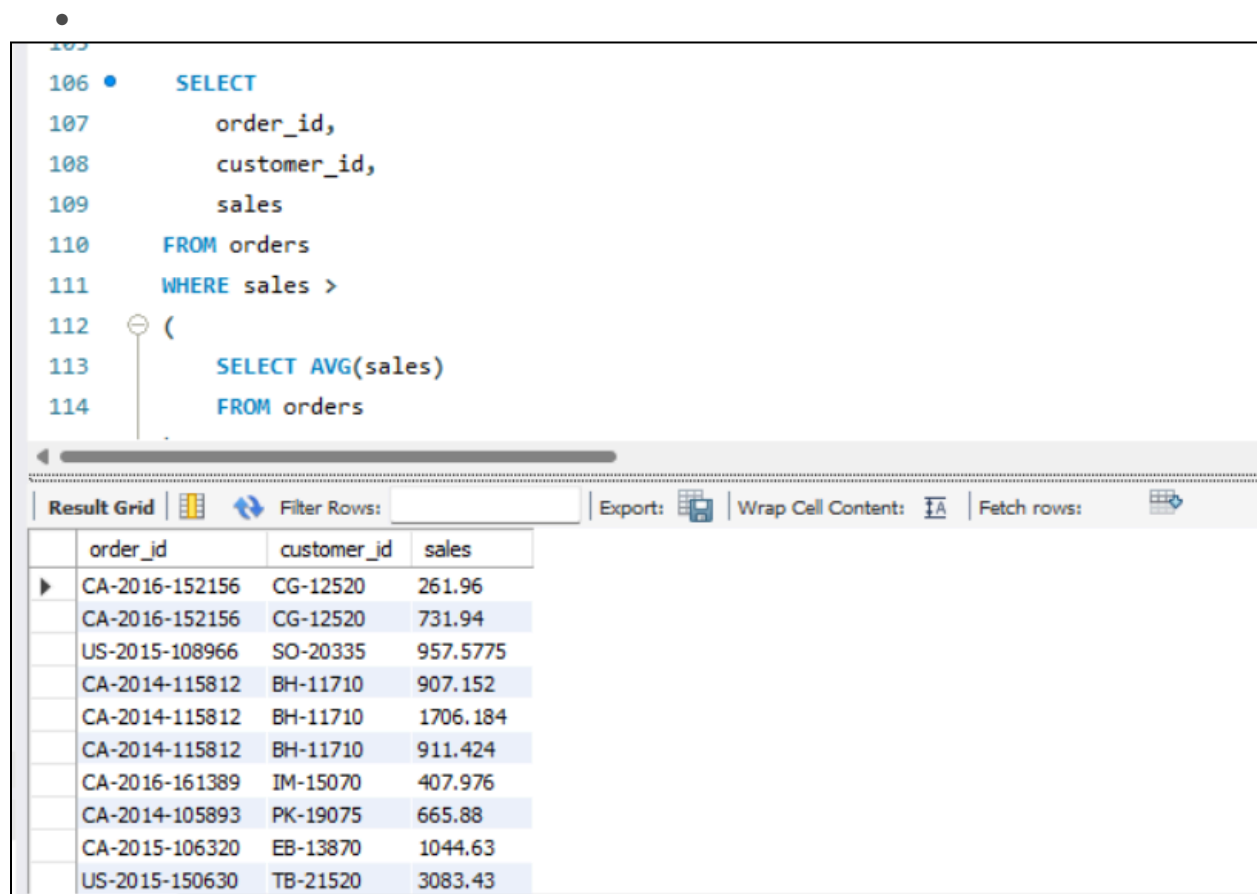


WEEK 3 ASSIGNMENT:

Use Subqueries, CTEs, and Window Functions to analyze sales data from the Superstore dataset.

- Objective: Analyze sales data using SQL by applying Subqueries, CTEs, and Window Functions to solve business queries. Steps: Load Superstore dataset into a table (superstore_raw). Create tables (customers, orders, products) from the dataset. Insert data into these tables using SELECT DISTINCT. Apply subqueries to filter data (above average sales, highest order per customer). Use CTEs to compute aggregations (total sales per customer). Apply window functions (ROW_NUMBER, RANK) for ranking and analysis. Combine JOIN + CTE + Window Functions for final result (customer, total sales, rank). Solve business queries (top customers, low customers, single-order customers, above-average sales). Output: SQL script / Notebook + query results + brief insights.



```
106 • SELECT
107     order_id,
108     customer_id,
109     sales
110 FROM orders
111 WHERE sales >
112 (
113     SELECT AVG(sales)
114     FROM orders
```

	order_id	customer_id	sales
▶	CA-2016-152156	CG-12520	261.96
	CA-2016-152156	CG-12520	731.94
	US-2015-108966	SO-20335	957.5775
	CA-2014-115812	BH-11710	907.152
	CA-2014-115812	BH-11710	1706.184
	CA-2014-115812	BH-11710	911.424
	CA-2016-161389	IM-15070	407.976
	CA-2014-105893	PK-19075	665.88
	CA-2015-106320	EB-13870	1044.63
	US-2015-150630	TB-21520	3083.43

this query identifies all orders with sales greater than the average order value

Result Grid						
		Filter Rows:			Export:	Wrap Cell Content:
	Field	Type	Null	Key	Default	Extra
►	Row ID	int	YES		NULL	
	Order ID	text	YES		NULL	
	Order Date	text	YES		NULL	
	Ship Date	text	YES		NULL	
	Ship Mode	text	YES		NULL	
	Customer ID	text	YES		NULL	
	Customer Name	text	YES		NULL	
	Segment	text	YES		NULL	
	Country	text	YES		NULL	
	City	text	YES		NULL	
	State	text	YES		NULL	
	Postal Code	int	YES		NULL	
	Region	text	YES		NULL	
	Product ID	text	YES		NULL	
	Category	text	YES		NULL	
	Sub-Category	text	YES		NULL	
	Product Name	text	YES		NULL	
	Sales	double	YES		NULL	
	Quantity	int	YES		NULL	
	Discount	double	YES		NULL	
	Profit	double	YES		NULL	

Result 4

```

106 • SELECT
107     order_id,
108     customer_id,
109     sales
110 FROM orders
111 WHERE sales >
112 (
113     SELECT AVG(sales)
114     FROM orders

```

Result Grid   Filter Rows: Export:  Wrap Cell Content:  Fetch rows: 

	order_id	customer_id	sales
▶	CA-2016-152156	CG-12520	261.96
	CA-2016-152156	CG-12520	731.94
	US-2015-108966	SO-20335	957.5775
	CA-2014-115812	BH-11710	907.152
	CA-2014-115812	BH-11710	1706.184
	CA-2014-115812	BH-11710	911.424
	CA-2016-161389	IM-15070	407.976
	CA-2014-105893	PK-19075	665.88
	CA-2015-106320	EB-13870	1044.63
	US-2015-150630	TB-21520	3083.43

```

133 (
134     SELECT
135         customer_id,
136         SUM(sales) AS total_sales
137     FROM orders
138     GROUP BY customer_id
139 )
140
141 SELECT *
142 FROM CustomerSales
143 ORDER BY total_sales DESC;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	customer_id	total_sales
▶	SM-20320	25043.05
	TC-20980	19017.847999999998
	RB-19360	15117.339
	TA-21385	14595.62
	AB-10105	14355.610999999997
	SC-20095	14142.333999999999
	KL-16645	14071.917
	HL-15040	12873.297999999999
	SE-20110	12209.438000000002
	CC-12370	12129.072

Result 18 ×

Output

145 -- 4. Find customers whose total sales are above average. (CTE + Subquery)

146

147 • WITH CustomerSales AS

148 (

149 SELECT

150 customer_id,

151 SUM(sales) AS total_sales

152 FROM orders

153 GROUP BY customer_id

154)

155

Result Grid Filter Rows: Export: Wrap Cell Content:

	customer_id	total_sales
▶	SM-20320	25043.05
	TC-20980	19017.847999999998
	RB-19360	15117.339
	TA-21385	14595.62
	AB-10105	14355.610999999997
	SC-20095	14142.333999999999
	KL-16645	14071.917
	HL-15040	12873.297999999999
	SE-20110	12209.438000000002
	CC-12370	12129.072
	TS-21070	11605.071

Result 19 x

```
167 -- 5. Rank all customers based on total sales. (Window Function)
```

```
168
```

```
169 • WITH CustomerSales AS
```

```
170 (
```

```
171     SELECT
```

```
172         customer_id,
```

```
173         SUM(sales) AS total_sales
```

```
174     FROM orders
```

```
175     GROUP BY customer_id
```

```
176 )
```

```
177
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	customer_id	total_sales	sales_rank
▶	SM-20320	25043.05	1
	TC-20980	19017.847999999998	2
	RB-19360	15117.339	3
	TA-21385	14595.62	4
	AB-10105	14355.610999999997	5
	SC-20095	14142.333999999999	6
	KL-16645	14071.917	7
	HL-15040	12873.297999999999	8
	SE-20110	12209.438000000002	9
	CC-12370	12129.072	10

```
183
```

```
184 -- 6. Assign row numbers to each order within a customer. (Window Function + PARTITION BY)
```

```
185
```

```
186 • SELECT
```

```
187     customer_id,
```

```
188     order_id,
```

```
189     order_date,
```

```
190     sales,
```

```
191     ROW_NUMBER() OVER (
```

```
192         PARTITION BY customer_id
```

```
193         ORDER BY order_date
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	customer_id	order_id	order_date	sales	order_number
▶	AA-10315	CA-2015-121391	10/4/2015	26.96	1
	AA-10315	CA-2016-103982	3/3/2016	41.72	2
	AA-10315	CA-2016-103982	3/3/2016	3930.072	3
	AA-10315	CA-2016-103982	3/3/2016	2.304	4
	AA-10315	CA-2016-103982	3/3/2016	431.976	5
	AA-10315	CA-2014-128055	3/31/2014	52.98	6
	AA-10315	CA-2014-128055	3/31/2014	673.568	7
	AA-10315	CA-2017-147039	6/29/2017	362.94	8
	AA-10315	CA-2017-147039	6/29/2017	11.54	9
	AA-10315	CA-2014-138100	9/15/2014	14.94	10

```

197 -- 7. Display top 3 customers based on total sales. (Window Function)
198 • WITH CustomerSales AS
199 (
200     SELECT
201         customer_id,
202         SUM(sales) AS total_sales
203     FROM orders

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	customer_id	total_sales	sales_rank
▶	SM-20320	25043.05	1
	TC-20980	19017.847999999998	2
	RB-19360	15117.339	3

```

19 /* Step 3: Final Combined Query
20 Write one final query that shows:
21 Customer Name
22 Total Sales
23 Rank
24 (Use JOIN + CTE + Window Function together)
25 */

```

```

27 • WITH CustomerSales AS

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	customer_name	total_sales	sales_rank
	Sean Miller	25043.05	1
	Tamara Chand	19017.847999999998	2
	Raymond Buch	15117.339	3
	Tom Ashbrook	14595.62	4
	Adrian Barton	14355.610999999997	5
	Sanjit Chand	14142.333999999999	6
	Ken Lonsdale	14071.917	7
	Hunter Lopez	12873.297999999999	8
	Sanjit Engle	12209.438000000002	9
	Christopher Conant	12129.072	10

```

244
245 -- Mini Project: Customer Sales Insights
246 -- 1. who are the top 5 customers
247 • WITH CustomerSales AS
248 (
249     SELECT
250         customer_id,
251         SUM(sales) AS total_sales

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [⌕](#)

	customer_name	total_sales
▶	Sean Miller	25043.05
	Tamara Chand	19017.847999999998
	Raymond Buch	15117.339
	Tom Ashbrook	14595.62
	Adrian Barton	14355.610999999997

```

264
265 -- who are the bottom 5
266 • WITH CustomerSales AS
267 (
268     SELECT
269         customer_id,
270         SUM(sales) AS total_sales
271     FROM orders
272     GROUP BY customer_id
273 )
274

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [⌕](#)

	customer_name	total_sales
▶	Thais Sissman	4.833
	Lela Donovan	5.304
	Mitch Gastineau	12.32
	Carl Jackson	16.52
	Roy Skaria	22.328


```

283
284 -- Which customers made only one order?
285 • SELECT
286     c.customer_name,
287     COUNT(DISTINCT o.order_id) AS total_orders
288 FROM orders o
289 JOIN customers c
290 ON o.customer_id = c.customer_id
291 GROUP BY o.customer_id, c.customer_name
292 HAVING COUNT(DISTINCT o.order_id) = 1;

```

Result Grid   Filter Rows: | Export:  | Wrap Cell Content: 

	customer_name	total_orders
▶	Anthony O'Donnell	1
	Anemone Ratner	1
	Carl Jackson	1
	Jenna Caffey	1
	Jocasta Rupert	1
	Lela Donovan	1
	Mitch Gastineau	1
	Patricia Hirasaki	1
	Ricardo Emerson	1
	Roland Murray	1

294 -- 4. Which Customers Have Above-Average Sales?

295 • WITH CustomerSales AS

296 (

297 SELECT

298 customer_id,

299 SUM(sales) AS total_sales

300 FROM orders

301 GROUP BY customer_id

302)

303

304 SELECT

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	customer_name	total_sales
▶	Sean Miller	25043.05
	Tamara Chand	19017.847999999998
	Raymond Buch	15117.339
	Tom Ashbrook	14595.62
	Adrian Barton	14355.610999999997
	Sanjit Chand	14142.333999999999
	Ken Lonsdale	14071.917
	Hunter Lopez	12873.297999999999
	Sanjit Engle	12209.438000000002
	Christopher Conant	12129.072

317 -- 5. What is the Highest Order Value Per Customer?

318

```
319 • SELECT
320     c.customer_name,
321     MAX(o.sales) AS highest_order_value
322 FROM orders o
323 JOIN customers c
324 ON o.customer_id = c.customer_id
325 GROUP BY o.customer_id, c.customer_name
326 ORDER BY highest_order_value DESC;
```

Result Grid   Filter Rows: Export:  Wrap Cell Content: 

	customer_name	highest_order_value
▶	Sean Miller	22638.48
	Tamara Chand	17499.95
	Raymond Buch	13999.96
	Tom Ashbrook	11199.968
	Hunter Lopez	10499.97
	Adrian Barton	9892.74
	Sanjit Chand	9449.95
	Bill Shonely	9099.93
	Sanjit Engle	8749.95
	Christopher Conant	8399.976
	Michael H	8187.65

Result 29 x